MarketPulse AI

The Complete Master Guide

Market Problems | Tech Stack | All APIs | Django Code | Token Saving

Rate Limiting | Pricing | Zerodha API | PAN+OTP Portfolio | Legal

India ka pehla AI-powered Retail Investor Intelligence Platform

Django + Claude AI + Free APIs | 7 Problems Solved | 4 Build Phases | Complete Production Code

v3.0 COMPLETE | 2025-26

MarketPulse AI

The Complete Master Guide

India ka pehla AI-powered Retail Investor Intelligence Platform

7 Problems Solved	4 Build Phases	100% Free APIs	Zerodha Integration	PAN+OTP Portfolio	Token Saving	Rate Limiting

Table of Contents

PART A: Market & Product Strategy

1. India Retail Investor — The Opportunity

India mein retail investing 2020 ke baad explosive growth pe hai. Zerodha, Groww ne crores ko market mein laaya. Lekin tools abhi bhi primitive hain — aur yahi tera opportunity hai.

Metric	2020	2024	2026 (Est.)
Demat accounts	4 crore	15 crore	20+ crore
Daily NSE retail volume	■8,000 Cr	■35,000 Cr	■55,000 Cr+
Retail in F&O; (losing)	71%	91%	93%+
Avg retail return vs Nifty	-3% underperform	-5%	Still losing
AI tools for retail	Almost zero	Very few	Massive gap exists

Tip: SEBI 2024: F&O; mein 93% retail traders lose money. Inhe proper tools chahiye — yahi tera platform hai.

2. The 7 Core Problems & AI Solutions

P1	Information Overload & Noise	Phase 1 MVP

Hazaron news roz — Telegram tips, YouTube videos, broker reports. Asli impactful news filter karna impossible.

✓ **Personalized AI news feed** — Sirf woh news jo user ke portfolio/sectors affect kare. Claude tags each: 'Affects TCS, IT sector, Bullish 78%'

✓ **Impact scoring** — Har news pe 1-10 importance score. 8+ pe push alert. Baaki background mein

✓ **Daily 5-min digest** — Subah 7am: top 3 news for YOUR portfolio, seedha impact, koi noise nahi

✓ **Source credibility filter** — Telegram pump-and-dump vs ET Markets — AI source credibility score

P2	Emotional Decisions — FOMO & Panic	Phase 2

Market 5% up → sab khareed lete. 5% down → sab bech dete. Emotion se driven, data se nahi.

✓ **FOMO alert with context** — Jab market 3%+ move kare, AI explain kare: kyon hua, sustainable hai ya nahi

✓ **Behavioral nudge** — 'Tune last 3 baar 52-week high pe kharida, average -12% mila. Aaj bhi wahi pattern hai.'

✓ **Decision journal** — Har buy/sell record karo. 30 din baad AI review: 'Woh decision sahi tha?'

✓ **Cooling off alert** — Panic selling detect: '3 stocks 1 hour mein becho → AI: Kya sach mein zaroorat hai?'

P3	Poor Quality Research	Phase 1 MVP

Broker reports biased, influencer hype. Average investor ke paas unbiased analysis nahi.

✓ **AI earnings analyzer** — Results aaye → Claude: EPS beat/miss, guidance, sector ripple, expected move

✓ **Policy impact engine** — Budget, RBI, Election → 'HDFC Bank ke liye +2.3% expected, Banking bullish 3 months'

✓ **Global cue chain** — US Fed hike → FII outflow → Midcap IT pressure → smallcap selloff. Full chain

✓ **Sectoral shift tracker** — EV policy, PLI, import duty → which sector gains/loses, quantified

P4	Time Constraint	Phase 1 MVP

9-to-6 job wale investors market monitor nahi kar sakte. Opportunities miss, losses notice nahi.

✓ **Smart async alerts ONLY** — Notification sirf tab jab action needed — daily max 3 alerts. No spam

✓ **After-hours summary** — Market close → 5-min digest: kya hua, kyon hua, kal ke liye kya expect

✓ **Weekend deep dive** — Saturday morning: portfolio review, week ki top events, next week risks

✓ **Voice-first (future)** — WhatsApp pe 'Mera portfolio kaisa hai?' → AI jawab de

P5 Regulatory & Scam Risks — Phase 3

Pump & dump, unregistered advisors, options losses. SEBI 2024: 93% F&O; retail lose.

✓ **Pump & dump detector** — Telegram tip aaya → AI: volume pattern, promoter history, microcap flags check

✓ **SEBI advisor check** — Kisi ko follow karna → SEBI registration verify, complaint history

✓ **F&O; risk simulator** — Options trade → worst case P&L;, max loss, probability of profit dikhao

✓ **Scam pattern library** — Common Indian scam patterns: penny stocks, IPO fraud, fake broker alerts

P6 Poor Portfolio & Risk Mgmt — Phase 3

Over-concentration in IT, EV, PSU themes. No diversification awareness.

✓ **Concentration analyzer** — '60% IT mein hai. US recession → expected 28% portfolio drop. Diversify?'

✓ **Correlation matrix** — All IT stocks → effectively ek hi stock. Correlation dikhao

✓ **AI rebalancing** — Quarterly: 'Yeh overweight, yeh underweight. Tax-efficient rebalancing plan'

✓ **Stress test** — '2008 jaisi crash aaye → portfolio -X%. FII ■10k cr nikale?'

P7 Tax & Compliance Complexity — Phase 4

LTCG, STCG, STT, new budget changes. Optimize karna average investor ke bas ki baat nahi.

✓ **Real-time tax calc** — 'Agar TCS aaj becho → LTCG ■12,400, STCG ■3,200. Net tax: ■15,600'

✓ **Tax harvesting alerts** — December: 'In 5 stocks mein loss hai. Book karo → ■18k tax save karo'

✓ **Budget impact** — Budget announce → AI: 'Aapke portfolio pe direct tax impact: +/-■X'

✓ **Compliance calendar** — Advance tax dates, ITR filing, ELSS deadlines → automated reminders

3. Competitive Landscape — Where You Win

Platform	What They Do	What's Missing	Your Advantage
Zerodha/Groww	Brokerage + basic charts	No AI, no news analysis	AI impact analysis
Moneycontrol/ET	News aggregation	Not personalized, no AI	Portfolio-linked news
Smallcase/Tickertape	Basket investing	No real-time alerts	Live AI sentiment
Pulse by Zerodha	Some news	Very basic, no Hindi	Hinglish AI chat
Bloomberg/Refinitiv	Institutional grade	■5L+/year, English only	Affordable + Hindi
MarketPulse AI	ALL OF THE ABOVE	—	This is you!

Tip: Tera moat: Hinglish + portfolio-personalized + AI-powered + affordable. Koi yeh combo nahi de raha 2026 mein.

4. Product Vision & Feature Map

Feature	Problem Solved	Phase	Priority
Personalized news feed	Overload	1	Critical
AI sentiment tagging	Overload, Research	1	Critical
Sector impact analysis	Research	1	Critical
Daily 5-min digest email	Time	1	High
Smart push alerts	Time	1	High
User watchlist	Time	1	High
FOMO alert with context	Emotions	2	High
AI earnings analyzer	Research	2	High
Hinglish AI chatbot	All problems	2	High
Decision journal	Emotions	2	Medium
Portfolio analyzer	Risk	3	High
Concentration risk alert	Risk	3	High
Pump & dump detector	Scam	3	Medium
F&O; risk simulator	Scam	3	Medium
Tax calculator	Tax	4	Medium
Tax harvesting alerts	Tax	4	Medium

PAN + OTP portfolio import	All	4	High
Zerodha OAuth sync	All	3	High

PART B: Tech Stack & Free APIs

5. Tech Stack & Architecture

Layer	Technology	Why This Choice
Backend	Django 5.x	Mature, batteries-included, ORM, free admin panel
Database	PostgreSQL	JSON fields, full-text search, reliable at scale
Task Queue	Celery + Redis	Auto-fetch news every 10 min, async AI processing
AI / NLP	Claude API (Haiku + Sonnet)	Best reasoning, Hindi support, affordable
Price Data	yfinance (Python)	100% free, NSE/BSE/Crypto all supported
News Sources	RSS Feeds + NewsAPI	Mix of free unlimited + freemium APIs
Live Updates	Django Channels (WebSocket)	Real-time updates without reload
Frontend	Django Templates + HTMX	No React needed, fast, simple, SEO-friendly
Charts	TradingView Widget (free)	Professional-grade charts, zero cost
Email	Gmail SMTP (free)	500 emails/day free for alerts
Alerts	Telegram Bot API (free)	Unlimited, instant, no cost ever
Cache	Redis	Fast serving, rate limit, token budget tracking
Hosting	Railway.app	Free tier, auto-deploy from GitHub
Payments	Razorpay	Indian payments, UPI support, easy integration

6. Free APIs — Complete Master List

Phase 1 mein **zero rupees** kharcha. Ye sab free tier mein MVP cover karti hain.

NEWS APIS

NewsAPI.org	Free	100 req/day	Top headlines, stocks & crypto
GNews API	Free	100 req/day	Google News aggregator
CurrentsAPI	Free	600 req/day	Best free tier — finance heavy
Mediastack	Free	500 req/month	Global, Indian sources
TheNewsAPI	Free	100 req/day	Finance & business focused

FREE RSS FEEDS — Unlimited, No Key

Source	RSS URL	Category
Economic Times Markets	economictimes.indiatimes.com/markets/rss.cms	Indian Stocks
Moneycontrol	moneycontrol.com/rss/latestnews.xml	Indian Markets
Livemint Markets	livemint.com/rss/markets	India Finance
Financial Express	financialexpress.com/market/feed/	Indian Stocks
Business Standard	business-standard.com/rss/markets-106.rss	India Markets
CoinDesk	coindesk.com/arc/outboundfeeds/rss/	Crypto Global
CoinTelegraph	cointelegraph.com/rss	Crypto News
Reuters Business	feeds.reuters.com/reuters/businessNews	Global
CNBC Markets	cnbc.com/id/20409666/device/rss/rss.html	Global
Investing.com India	in.investing.com/rss/news.rss	India + Global

MARKET DATA — Free

yfinance (Python)	Free	Unlimited	NSE/BSE/Crypto prices, no key needed!
Alpha Vantage	Free	25 req/day	Stock prices, technical indicators
CoinGecko API	Free	30 calls/min	Crypto prices, market cap
Binance API	Free	1200 req/min	Crypto OHLCV, order book
NSE India (nsetools)	Free	Unlimited	Python library, NSE live data

ALERTS & NOTIFICATIONS — Free

Telegram Bot API	Free	Unlimited	Best free alert channel — instant
Gmail SMTP	Free	500/day	Email alerts, digest, reports
Fast2SMS	Free	50 SMS/day	OTP verification for PAN flow
Firebase FCM	Free	Unlimited	Mobile push notifications

FINANCIAL DATA — Free

Screener.in	Free	Reasonable	Fundamental data — P/E, ROE, revenue
NSE India website	Free	Public	Bulk deals, FII/DII data
SEBI website	Free	Public	Registered advisor list
RBI website RSS	Free	Unlimited	Rate decisions, policy
Surepass PAN verify	Paid	■2-3/verify	PAN verification for portfolio import

PART C: Build Guide — Phase by Phase

7. Django Project Setup

Step 1 — Install All Dependencies

```
python -m venv venv && source venv/bin/activate
pip install django==5.0 celery redis django-celery-beat
pip install feedparser requests python-dotenv anthropic
pip install yfinance django-allauth channels pillow
pip install whitenoise psycopg2-binary django-redis
pip install python-telegram-bot razorpay kiteconnect
pip install cryptography sentry-sdk
```

Step 2 — Create Project & Apps

```
django-admin startproject marketpulse .
python manage.py startapp news
python manage.py startapp sentiment
python manage.py startapp watchlist
python manage.py startapp alerts
python manage.py startapp portfolio
python manage.py startapp dashboard
```

Step 3 — settings.py

```
import os; from pathlib import Path; from dotenv import load_dotenv
load_dotenv()
BASE_DIR=Path(__file__).resolve().parent.parent
SECRET_KEY=os.environ.get('SECRET_KEY','dev-key')
DEBUG=os.environ.get('DEBUG','True')=='True'
ALLOWED_HOSTS=os.environ.get('ALLOWED_HOSTS','localhost').split(',')

INSTALLED_APPS=['django.contrib.admin','django.contrib.auth','django.contrib.contenttypes',
    'django.contrib.sessions','django.contrib.messages','django.contrib.staticfiles',
    'allauth','allauth.account','channels',
    'news','sentiment','watchlist','alerts','portfolio','dashboard']

DATABASES={'default':{'ENGINE':'django.db.backends.postgresql',
    'NAME':os.environ.get('DB_NAME','marketpulse'),
    'USER':os.environ.get('DB_USER','postgres'),
    'PASSWORD':os.environ.get('DB_PASSWORD',''),
    'HOST':os.environ.get('DB_HOST','localhost'),'PORT':'5432'}}

CELERY_BROKER_URL=os.environ.get('REDIS_URL','redis://localhost:6379/0')
CELERY_RESULT_BACKEND=os.environ.get('REDIS_URL','redis://localhost:6379/0')
CELERY_BEAT_SCHEDULE={
    'fetch-news-10min':{'task':'news.tasks.fetch_all_news','schedule':600.0},
    'morning-digest-7am':{'task':'alerts.tasks.send_morning_digest','schedule':25200.0},
    'price-alerts-5min':{'task':'alerts.tasks.check_price_alerts','schedule':300.0},
    'fomo-detect-15min':{'task':'sentiment.tasks.detect_fomo_events','schedule':900.0},
    'review-decisions-daily':{'task':'portfolio.tasks.review_old_decisions','schedule':86400.0},
}
ANTHROPIC_API_KEY=os.environ.get('ANTHROPIC_API_KEY','')
TELEGRAM_BOT_TOKEN=os.environ.get('TELEGRAM_BOT_TOKEN','')
```

```
NEWS_API_KEY=os.environ.get('NEWS_API_KEY','')
ZERODHA_API_KEY=os.environ.get('ZERODHA_API_KEY','')
ZERODHA_API_SECRET=os.environ.get('ZERODHA_API_SECRET','')
RAZORPAY_KEY_ID=os.environ.get('RAZORPAY_KEY_ID','')
RAZORPAY_KEY_SECRET=os.environ.get('RAZORPAY_KEY_SECRET','')
SUREPASS_TOKEN=os.environ.get('SUREPASS_TOKEN','')
FAST2SMS_API_KEY=os.environ.get('FAST2SMS_API_KEY','')
PAN_ENCRYPTION_KEY=os.environ.get('PAN_ENCRYPTION_KEY','')
EMAIL_BACKEND='django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST='smtp.gmail.com'; EMAIL_PORT=587; EMAIL_USE_TLS=True
EMAIL_HOST_USER=os.environ.get('EMAIL_USER','')
EMAIL_HOST_PASSWORD=os.environ.get('EMAIL_PASS','')
CACHES={'default':{'BACKEND':'django_redis.cache.RedisCache',
    'LOCATION':os.environ.get('REDIS_URL','redis://localhost:6379/1')}}
STATIC_URL='/static/'; STATIC_ROOT=BASE_DIR/'staticfiles'
STATICFILES_STORAGE='whitenoise.storage.CompressedManifestStaticFilesStorage'
```

8. Phase 1 — News Aggregation + Sentiment MVP

PHASE 1 News Fetch + AI Sentiment + Dashboard

8.1 News Models (news/models.py)

```
from django.db import models
class NewsSource(models.Model):
    CATS=[('crypto','Crypto'),('stocks','Stocks'),('macro','Macro'),('global','Global')]
    name=models.CharField(max_length=100); url=models.URLField()
    category=models.CharField(max_length=20,choices=CATS)
    credibility_score=models.FloatField(default=0.7)
    is_rss=models.BooleanField(default=True); is_active=models.BooleanField(default=True)

class NewsArticle(models.Model):
    source=models.ForeignKey(NewsSource,on_delete=models.CASCADE)
    title=models.CharField(max_length=500); url=models.URLField(unique=True)
    summary=models.TextField(blank=True); published=models.DateTimeField()
    fetched_at=models.DateTimeField(auto_now_add=True)
    is_analyzed=models.BooleanField(default=False)
    class Meta:
        ordering=['-published']
        indexes=[models.Index(fields=['-published']),models.Index(fields=['is_analyzed'])]
```

8.2 Sentiment Model (sentiment/models.py)

```
from django.db import models
from news.models import NewsArticle
class SentimentResult(models.Model):
    SENT=[('bullish','Bullish'),('bearish','Bearish'),('neutral','Neutral')]
    article=models.OneToOneField(NewsArticle,on_delete=models.CASCADE,related_name='sentiment')
    sentiment=models.CharField(max_length=10,choices=SENT)
    confidence=models.FloatField(default=0.0); importance=models.IntegerField(default=5)
    summary_hindi=models.TextField(blank=True)
    stocks=models.JSONField(default=list); sectors=models.JSONField(default=list)
    expected_move=models.JSONField(default=dict); time_horizon=models.CharField(max_length=20,default='1-3 d
ays')
    analyzed_at=models.DateTimeField(auto_now_add=True)
```

8.3 News Fetcher (news/tasks.py)

```
import feedparser,requests
from celery import shared_task
from django.utils import timezone
from .models import NewsSource,NewsArticle

@shared_task
def fetch_all_news():
    total=0
    for source in NewsSource.objects.filter(is_active=True):
        try:
            feed=feedparser.parse(source.url)
            for entry in feed.entries[:25]:
                art,created=NewsArticle.objects.get_or_create(
                    url=getattr(entry,'link',''),
                    defaults={'source':source,'title':entry.title[:500],
                              'summary':getattr(entry,'summary','')[:2000],'published':timezone.now()})
                if created:
```

```
                    total+=1
                    from sentiment.tasks import analyze_article
                    analyze_article.delay(art.id)
        except Exception as e: print(f"Error {source.name}: {e}")
    return f"Fetched {total} new articles"
```

8.4 Claude AI Analyzer (sentiment/analyzer.py)

```
import anthropic,json
from django.conf import settings
client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

def analyze_batch(articles:list)->list:
    """10 articles = 1 Claude call = 85% cost saving!"""
    items="
".join([f"{i+1}|{a.title[:120]}" for i,a in enumerate(articles[:10])])
    prompt=f"""Analyze {len(articles[:10])} Indian market headlines.
Return ONLY JSON array, no explanation:
{items}
Format: [{{"idx":1,"s":"bullish","i":7,"stocks":["TCS"],"sec":["IT"],"m":"+1.5%","h":"1-3 days",
"summary":"RBI ne rates ghatayi — Banking ke liye bullish hai."}}]
s=sentiment,i=importance(1-10),stocks=NSE tickers,sec=sectors,m=expected_move,h=time_horizon,summary=Hinglis
h"""
    msg=client.messages.create(model="claude-haiku-20240307",max_tokens=800,
        messages=[{"role":"user","content":prompt}])
    try: return json.loads(msg.content[0].text)
    except: return []

def analyze_single(title:str,summary:str)->dict:
    """Single article — use only for high-importance news"""
    prompt=f"""Analyze. JSON only:
Title: {title[:150]}
{{"s":"bullish/bearish/neutral","i":1-10,"stocks":[],"sec":[],"m":"","h":"","summary":""}}"""
    msg=client.messages.create(model="claude-haiku-20240307",max_tokens=250,
        messages=[{"role":"user","content":prompt}])
    return json.loads(msg.content[0].text)
```

8.5 Analysis Task (sentiment/tasks.py)

```
from celery import shared_task
from news.models import NewsArticle
from .models import SentimentResult
from .analyzer import analyze_batch
import hashlib
from django.core.cache import cache

SKIP_KEYWORDS=['cricket','ipl','bollywood','film','weather','recipe','fashion','sports score']
PRIORITY_KW=['rbi','sebi','nifty','sensex','result','earnings','merger','ipo','fii','rate','budget','gdp']

def should_analyze(article)->tuple:
    title_lower=article.title.lower()
    if any(k in title_lower for k in SKIP_KEYWORDS): return False,0
    dup_key="dup_"+hashlib.md5(title_lower[:60].encode()).hexdigest()[:12]
    if cache.get(dup_key): return False,0
    cache.set(dup_key,True,3600)
    priority=5
    for k in PRIORITY_KW:
        if k in title_lower: priority=min(10,priority+1)
    return True,priority

@shared_task
def process_pending_articles():
    pending=list(NewsArticle.objects.filter(is_analyzed=False).order_by('-published')[:50])
    for i in range(0,len(pending),10):
        batch=pending[i:i+10]
```

```
        filtered=[a for a in batch if should_analyze(a)[0]]
        if not filtered: continue
        results=analyze_batch(filtered)
        if results: save_batch_results.delay([a.id for a in filtered],results)

@shared_task
def save_batch_results(article_ids:list,results:list):
    for r in results:
        idx=r.get('idx',0)-1
        if 0<=idx           try:
                article=NewsArticle.objects.get(id=article_ids[idx])
                SentimentResult.objects.get_or_create(article=article,defaults={
                    'sentiment':r.get('s','neutral'),'confidence':0.8,
                    'importance':r.get('i',5),'stocks':r.get('stocks',[]),
                    'sectors':r.get('sec',[]),'expected_move':{'move':r.get('m','')},
                    'time_horizon':r.get('h','1-3 days'),'summary_hindi':r.get('summary','')})
                article.is_analyzed=True; article.save(update_fields=['is_analyzed'])
            except: pass

@shared_task
def detect_fomo_events():
    import yfinance as yf
    from watchlist.models import UserWatchlist
    symbols=UserWatchlist.objects.values_list('stock__symbol',flat=True).distinct()
    for symbol in symbols:
        try:
            ticker=yf.Ticker(f"{symbol}.NS")
            hist=ticker.history(period="2d",interval="1h")
            if len(hist)<2: continue
            pct=((hist['Close'].iloc[-1]/hist['Close'].iloc[-2])-1)*100
            if abs(pct)>=3.0:
                from alerts.tasks import send_fomo_alert
                send_fomo_alert.delay(symbol,round(pct,2))
        except: pass
```

9. Phase 2 — AI Intelligence Layer + Chatbot

PHASE 2 FOMO Alerts + Hinglish Chatbot + Decision Journal

9.1 FOMO Alert with AI Context (alerts/tasks.py)

```
from celery import shared_task
from django.core.mail import send_mail
from django.conf import settings
import anthropic,telegram

client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

@shared_task
def send_fomo_alert(symbol:str,pct_change:float):
    direction="upar" if pct_change>0 else "neeche"
    msg=client.messages.create(model="claude-haiku-20240307",max_tokens=150,
        messages=[{"role":"user","content":f"{symbol} {abs(pct_change):.1f}% {direction} gaya. "
            f"50 words Hinglish mein: (1) probable reason, (2) sustainable hai?, (3) kya karna chahiye?"}])
    ai_ctx=msg.content[0].text
    emoji="?" if pct_change>0 else "?"
    message=f"{emoji} {symbol} Alert: {pct_change:+.1f}%

AI: {ai_ctx}

—Emotion pe mat karo React!"
    from watchlist.models import UserWatchlist
    for w in UserWatchlist.objects.filter(stock__symbol=symbol,alert_on_mention=True).select_related('user__
profile'):
        profile=getattr(w.user,'profile',None)
        if profile and profile.telegram_chat_id:
            try: telegram.Bot(token=settings.TELEGRAM_BOT_TOKEN).send_message(chat_id=profile.telegram_chat_
id,text=message)
            except: pass
        if w.user.email:
            send_mail(f"MarketPulse: {symbol} {pct_change:+.1f}%",message,'alerts@marketpulse.ai',[w.user.em
ail],fail_silently=True)

@shared_task
def send_morning_digest():
    from django.contrib.auth.models import User
    from sentiment.models import SentimentResult
    from django.utils import timezone; from datetime import timedelta
    yesterday=timezone.now()-timedelta(hours=24)
    for user in User.objects.filter(is_active=True):
        watchlist=list(user.userwatchlist_set.values_list('stock__symbol',flat=True))
        if not watchlist: continue
        recent=SentimentResult.objects.filter(analyzed_at__gte=yesterday,importance__gte=7).select_related('
article').order_by('-importance')[:5]
        relevant=[r for r in recent if any(s in r.stocks for s in watchlist)][:3]
        if not relevant: continue
        lines=[f"Namaste {user.first_name or 'Investor'}! Aaj ki top news:
"]
        for i,r in enumerate(relevant,1):
            e="?" if r.sentiment=='bullish' else "?" if r.sentiment=='bearish' else "?"
            lines.append(f"{i}. {e} {r.summary_hindi}
   Stocks: {', '.join(r.stocks[:3])}")
        lines.append("
Happy Investing! — MarketPulse AI")
```

```
        profile=getattr(user,'profile',None)
        if profile and profile.telegram_chat_id:
            try: telegram.Bot(token=settings.TELEGRAM_BOT_TOKEN).send_message(chat_id=profile.telegram_chat_
id,text="
".join(lines))
            except: pass
```

9.2 Hinglish AI Chatbot (dashboard/views.py)

```
import json,anthropic
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from django.contrib.auth.decorators import login_required
from django.conf import settings
from utils.rate_limit import feature_required

client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

@csrf_exempt
@login_required
@feature_required('chat')
def ai_chat(request):
    if request.method!='POST': return JsonResponse({'error':'POST only'})
    data=json.loads(request.body)
    user_msg=data.get('message',''); history=data.get('history',[])
    watchlist=list(request.user.userwatchlist_set.values_list('stock__symbol',flat=True))
    from sentiment.models import SentimentResult
    recent=SentimentResult.objects.select_related('article').order_by('-analyzed_at')[:8]
    ctx="
".join([f"- {r.article.title}: {r.sentiment}, {r.stocks}" for r in recent])
    system=f"""Aap MarketPulse AI hain — India ka smartest market assistant.
User watchlist: {', '.join(watchlist) or 'Not set'}
Recent market context:
{ctx}
Rules: Hinglish mein jawab, max 150 words, data-backed, no FOMO, always add disclaimer."""
    resp=client.messages.create(model="claude-haiku-20240307",max_tokens=300,system=system,
        messages=history+[{"role":"user","content":user_msg}])
    return JsonResponse({'reply':resp.content[0].text,'disclaimer':'Yeh investment advice nahi hai.'})

@csrf_exempt
@login_required
@feature_required('earnings')
def earnings_analyzer(request):
    if request.method!='POST': return JsonResponse({'error':'POST only'})
    company=json.loads(request.body).get('company','')
    prompt=f"""{company} quarterly results. Hinglish mein:
1. EPS beat/miss by how much
2. Revenue/EBITDA/PAT YoY%
3. Management guidance
4. Expected stock move next 1-3 months
5. Hold/Sell/Accumulate recommendation
Note: Use sector patterns if exact data unavailable."""
    msg=client.messages.create(model="claude-sonnet-4-20250514",max_tokens=500,
        messages=[{"role":"user","content":prompt}])
    return JsonResponse({'analysis':msg.content[0].text})
```

9.3 Decision Journal (portfolio/models.py)

```
from django.db import models
from django.contrib.auth.models import User

class TradeDecision(models.Model):
    ACTION=[('buy','Buy'),('sell','Sell'),('hold','Hold')]
    user=models.ForeignKey(User,on_delete=models.CASCADE)
    stock=models.CharField(max_length=20); action=models.CharField(max_length=10,choices=ACTION)
```

```
    price=models.DecimalField(max_digits=10,decimal_places=2); quantity=models.IntegerField()
    reason=models.TextField(); emotion=models.CharField(max_length=20,blank=True)
    decided_at=models.DateTimeField(auto_now_add=True)
    review_price=models.DecimalField(max_digits=10,decimal_places=2,null=True)
    review_pnl_pct=models.FloatField(null=True)
    ai_review=models.TextField(blank=True); reviewed_at=models.DateTimeField(null=True)

@shared_task
def review_old_decisions():
    import yfinance as yf; import anthropic
    from django.utils import timezone; from datetime import timedelta
    cutoff=timezone.now()-timedelta(days=30)
    client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)
    for d in TradeDecision.objects.filter(decided_at__lte=cutoff,reviewed_at__isnull=True):
        try:
            current=yf.Ticker(f"{d.stock}.NS").info.get('currentPrice',0)
            if not current: continue
            pnl=((current-float(d.price))/float(d.price))*100
            if d.action=='sell': pnl=-pnl
            msg=client.messages.create(model="claude-haiku-20240307",max_tokens=150,
                messages=[{"role":"user","content":f"30 din pehle {d.stock} {d.action} at {d.price}. "
                    f"Reason: '{d.reason}'. Emotion: {d.emotion}. Aaj: {current:.0f}. P&L;: {pnl:+.1f}%. "
                    f"3 sentences Hinglish: decision sahi tha?, emotion ne help/hurt kiya?, 1 lesson."}])
            d.review_price=current; d.review_pnl_pct=round(pnl,2)
            d.ai_review=msg.content[0].text; d.reviewed_at=timezone.now(); d.save()
        except: pass
```

10. Phase 3 — Portfolio + Risk + Scam Detection

PHASE 3 **Portfolio Intelligence + Concentration Risk + Pump & Dump**

10.1 Watchlist & Portfolio Models (watchlist/models.py)

```
from django.db import models
from django.contrib.auth.models import User

class Stock(models.Model):
    symbol=models.CharField(max_length=20,unique=True)
    name=models.CharField(max_length=200); exchange=models.CharField(max_length=10,default='NSE')
    sector=models.CharField(max_length=50,blank=True)
    def get_price(self):
        from django.core.cache import cache
        cached=cache.get(f'price_{self.symbol}')
        if cached: return cached
        import yfinance as yf
        info=yf.Ticker(f"{self.symbol}.NS").info
        data={'price':info.get('currentPrice',0),'change_pct':info.get('regularMarketChangePercent',0)}
        cache.set(f'price_{self.symbol}',data,300); return data

class UserWatchlist(models.Model):
    user=models.ForeignKey(User,on_delete=models.CASCADE)
    stock=models.ForeignKey(Stock,on_delete=models.CASCADE)
    quantity=models.IntegerField(default=0); avg_price=models.DecimalField(max_digits=10,decimal_places=2,nu
ll=True)
    alert_on_mention=models.BooleanField(default=True)
    alert_price_up=models.FloatField(null=True); alert_price_down=models.FloatField(null=True)
    source=models.CharField(max_length=20,default='manual')
    added_at=models.DateTimeField(auto_now_add=True)
    class Meta: unique_together=('user','stock')
    @property
    def current_pnl_pct(self):
        if not self.quantity or not self.avg_price: return None
        current=self.stock.get_price().get('price',0)
        return round(((current-float(self.avg_price))/float(self.avg_price))*100,2)
```

10.2 Portfolio Risk Analyzer (portfolio/analyzer.py)

```
import anthropic,json
from django.conf import settings
client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

def analyze_portfolio_risk(user)->dict:
    from watchlist.models import UserWatchlist
    holdings=UserWatchlist.objects.filter(user=user,quantity__gt=0).select_related('stock')
    if not holdings: return {'error':'Portfolio empty'}
    total_value=0; sector_exp={}; portfolio_data=[]
    for h in holdings:
        price=h.stock.get_price().get('price',0)
        value=h.quantity*price; total_value+=value
        sector=h.stock.sector or 'Unknown'
        sector_exp[sector]=sector_exp.get(sector,0)+value
        portfolio_data.append({'symbol':h.stock.symbol,'sector':sector,'value':round(value,2),'pnl':h.curren
t_pnl_pct})
    sector_pcts={s:round(v/total_value*100,1) for s,v in sector_exp.items()} if total_value else {}
    prompt=f"""Indian portfolio analyze karo:
Holdings: {json.dumps(portfolio_data[:15])}
```

```
Sectors: {json.dumps(sector_pcts)}
Total: Rs {total_value:,.0f}
JSON only: {{"overall_risk":"High/Medium/Low","risk_score":7.2,
"concentration_risk":"IT 65% — concentrated",
"top_risks":["risk1","risk2"],"suggested_actions":["action1"],
"stress_test":{{"market_crash_20pct":"Portfolio -28%","it_crash":"Portfolio -18%"}},
"diversification_score":4,"verdict_hindi":"Aapka portfolio..."}}"""
    msg=client.messages.create(model="claude-sonnet-4-20250514",max_tokens=500,
        messages=[{"role":"user","content":prompt}])
    return json.loads(msg.content[0].text)
```

10.3 Pump & Dump Detector (sentiment/scam_detector.py)

```
import anthropic,json,yfinance as yf
from django.conf import settings
client=anthropic.Anthropic(api_key=settings.ANTHROPIC_API_KEY)

def detect_pump_dump(symbol:str,tip_text:str="")->dict:
    try:
        hist=yf.Ticker(f"{symbol}.NS").history(period="30d")
        info=yf.Ticker(f"{symbol}.NS").info
        vol_spike=hist['Volume'].tail(5).mean()/hist['Volume'].head(20).mean()
        price_30d=((hist['Close'].iloc[-1]/hist['Close'].iloc[0])-1)*100
        mktcap=info.get('marketCap',0); is_micro=mktcap<500_000_000
    except: vol_spike,price_30d,mktcap,is_micro=1,0,0,True
    prompt=f"""Pump-and-dump risk assess karo:
Stock:{symbol} Vol spike:{vol_spike:.1f}x Price 30d:{price_30d:.1f}% Mktcap:Rs{mktcap/10000000:.0f}Cr Microc
ap:{is_micro}
Tip:"{tip_text[:150]}"
JSON: {{"risk_level":"High/Medium/Low","red_flags":[],"verdict_hindi":"...","recommendation":"...","pump_pro
b":0.75}}"""
    msg=client.messages.create(model="claude-haiku-20240307",max_tokens=250,
        messages=[{"role":"user","content":prompt}])
    return json.loads(msg.content[0].text)
```

11. Phase 4 — Tax Intelligence + Compliance

PHASE 4 **Tax Calculator + Harvesting + Compliance Calendar**

```
# portfolio/tax.py
def calculate_capital_gains(buy_price,sell_price,quantity,buy_date,sell_date):
    holding_days=(sell_date-buy_date).days
    is_ltcg=holding_days>=365
    gain=(sell_price-buy_price)*quantity
    if gain<=0:
        return {'gain':gain,'tax':0,'type':'Loss','tax_saved_if_booked':abs(gain)*0.20}
    if is_ltcg:
        taxable=max(0,gain-100000)  # Rs 1L exempt
        tax=taxable*0.10  # LTCG 10% (Budget 2024)
        tip=f'{365-holding_days} din aur → LTCG rate!' if not is_ltcg else 'Rs 1L exempt hai LTCG mein!'
    else:
        tax=gain*0.20  # STCG 20% (Budget 2024 updated from 15%)
        tip=f'{365-holding_days} din aur hold karo → LTCG rate milega'
    return {'gain':round(gain,2),'tax':round(tax,2),'type':'LTCG' if is_ltcg else 'STCG',
            'holding_days':holding_days,'tip':tip}

def find_harvesting_opportunities(user):
    from watchlist.models import UserWatchlist
    opps=[]
    for h in UserWatchlist.objects.filter(user=user,quantity__gt=0).select_related('stock'):
        if not h.avg_price: continue
        current=h.stock.get_price().get('price',0)
        if current            loss=(float(h.avg_price)-current)*h.quantity
            opps.append({'symbol':h.stock.symbol,'unrealized_loss':round(loss,2),
                         'tax_saving':round(loss*0.20,2)})
    return sorted(opps,key=lambda x:x['tax_saving'],reverse=True)
```

PART D: Token Saving & Rate Limiting

12. Claude API Token Saving — 80% Cost Reduction

Strategy	Savings	Complexity	Implement When
Batch processing (10 articles/1 call)	85%	Low	Day 1 — MUST
Use Haiku instead of Sonnet	75%	Very Low	Day 1 — MUST
Redis cache AI results 24h	60%	Low	Day 1
Filter unimportant articles	40%	Low	Day 1
Short prompt engineering	20%	Low	Day 1
Duplicate title detection	25%	Low	Week 2
User-triggered analysis only	50%	Medium	Phase 2

12.1 Model Selection Guide

Task	Model	Cost/1M tokens	Rationale
Batch news sentiment	claude-haiku-20240307	$0.25 in / $1.25 out	Bulk processing — cheapest
Chatbot replies	claude-haiku-20240307	$0.25 in / $1.25 out	Fast + cheap for Q&A;
FOMO alert context	claude-haiku-20240307	$0.25 in / $1.25 out	Short context generation
Scam detection	claude-haiku-20240307	$0.25 in / $1.25 out	Pattern matching task
Portfolio risk analysis	claude-sonnet-4-20250514	$3 in / $15 out	Needs deep reasoning
Earnings deep dive	claude-sonnet-4-20250514	$3 in / $15 out	Accuracy critical
Tax analysis	claude-sonnet-4-20250514	$3 in / $15 out	Complex calculations

12.2 Redis Cache for AI Results

```
# sentiment/cache.py
import hashlib,json
from django.core.cache import cache

def get_or_analyze(title:str,summary:str,analyzer_fn)->dict:
    """Cache hit = free! No API call."""
    key="ai_"+hashlib.md5(title.lower()[:80].encode()).hexdigest()[:16]
```

```
cached=cache.get(key)
if cached: return cached  # FREE — no API call!
result=analyzer_fn(title,summary)
cache.set(key,result,86400)  # Cache 24 hours
return result
```

12.3 Daily Token Budget

Task	Calls/Day	Tokens/Call	Daily Cost (INR)
Batch news (10/call)	50 batches	800	~■4
Chatbot replies	100	300	~■3
FOMO alerts	20	200	~■0.4
Morning digest	500 users	400	~■20
Portfolio analysis (Sonnet)	10	1500	~■4.5
Earnings deep dive (Sonnet)	5	2000	~■4.5
TOTAL	685 calls	—	~■37/day = ■1,100/month

Tip: ■1,100/month mein 500 users ka platform chalega. 500 paid users = ■1L+/month revenue. ROI = 90x!

13. Redis Rate Limiting — Complete Implementation

13.1 Core Rate Limiter (utils/rate_limit.py)

```
from django.core.cache import cache
from django.http import JsonResponse
from functools import wraps
import time

PLAN_LIMITS={
    "free":    {"chat":{"limit":10,"window":86400},"portfolio":{"limit":1,"window":86400},
                "earnings":{"limit":2,"window":86400},"scam_check":{"limit":3,"window":86400},
                "tax_calc":{"limit":0,"window":86400},"news_refresh":{"limit":20,"window":3600}},
    "starter": {"chat":{"limit":50,"window":86400},"portfolio":{"limit":5,"window":86400},
                "earnings":{"limit":10,"window":86400},"scam_check":{"limit":20,"window":86400},
                "tax_calc":{"limit":5,"window":86400},"news_refresh":{"limit":100,"window":3600}},
    "pro":     {"chat":{"limit":200,"window":86400},"portfolio":{"limit":20,"window":86400},
                "earnings":{"limit":50,"window":86400},"scam_check":{"limit":100,"window":86400},
                "tax_calc":{"limit":30,"window":86400},"news_refresh":{"limit":500,"window":3600}},
    "trader":  {"chat":{"limit":9999,"window":86400},"portfolio":{"limit":9999,"window":86400},
                "earnings":{"limit":9999,"window":86400},"scam_check":{"limit":9999,"window":86400},
                "tax_calc":{"limit":9999,"window":86400},"news_refresh":{"limit":9999,"window":3600}},
}

def is_allowed(key:str,limit:int,window:int)->dict:
    count_key=f"rl:count:{key}"; reset_key=f"rl:reset:{key}"
    current=cache.get(count_key,0); reset_time=cache.get(reset_key)
    if current>=limit:
        reset_in=int(reset_time-time.time()) if reset_time else window
        return {'allowed':False,'remaining':0,'reset_in':max(0,reset_in)}
    if current==0:
        cache.set(count_key,1,window); cache.set(reset_key,time.time()+window,window)
    else: cache.incr(count_key)
    return {'allowed':True,'remaining':limit-current-1,'reset_in':0}

def get_user_plan(user)->str:
    try: return user.subscription.plan
    except: return 'free'

def check_feature(user,feature:str)->dict:
    plan=get_user_plan(user)
    config=PLAN_LIMITS.get(plan,PLAN_LIMITS['free']).get(feature,{"limit":0,"window":86400})
    if config['limit']==0:
        return {'allowed':False,'reason':f'Yeh feature {plan.title()} plan mein nahi hai.'}
    result=is_allowed(f"{feature}:user:{user.id}",config['limit'],config['window'])
    if not result['allowed']:
        result['reason']=f"Daily limit {config['limit']} reach ho gayi. Upgrade karo!"
    return result

def feature_required(feature:str):
    """One-line decorator for any view!"""
    def decorator(view_func):
        @wraps(view_func)
        def wrapped(request,*args,**kwargs):
            if not request.user.is_authenticated:
                return JsonResponse({'error':'Login required'},status=401)
            result=check_feature(request.user,feature)
            if not result['allowed']:
                return JsonResponse({'error':result.get('reason','Limit exceeded'),
                    'reset_in':result.get('reset_in',0),'message':'Plan upgrade karo!'},status=429)
            return view_func(request,*args,**kwargs)
```

```
        return wrapped
    return decorator

# Usage — sirf ek line:
# @feature_required('chat')    → chat rate limit
# @feature_required('portfolio') → portfolio rate limit
# @feature_required('earnings')  → earnings rate limit
```

PART E: Portfolio Integration

14. Pricing Strategy + Razorpay

Feature	Free ■0	Starter ■199	Pro ■499	Trader ■999
Watchlist stocks	5	20	100	Unlimited
AI Chat queries	10/day	50/day	200/day	Unlimited
Portfolio analysis	—	1/day	20/day	Unlimited
Earnings analyzer	—	10/day	50/day	Unlimited
FOMO alerts	3/day	15/day	Unlimited	Unlimited
Scam detector	3/day	20/day	100/day	Unlimited
Tax calculator	—	5/day	30/day	Unlimited
Zerodha OAuth sync	—	—	Yes	Yes
PAN portfolio import	—	—	—	Yes
API access	—	—	—	Yes

14.1 Subscription Model + Razorpay (users/models.py + payments/views.py)

```
# users/models.py
class Subscription(models.Model):
    PLANS=[('free','Free'),('starter','Starter'),('pro','Pro'),('trader','Trader')]
    user=models.OneToOneField(User,on_delete=models.CASCADE,related_name='subscription')
    plan=models.CharField(max_length=20,choices=PLANS,default='free')
    status=models.CharField(max_length=20,default='active')
    expires_at=models.DateTimeField(null=True)
    razorpay_sub_id=models.CharField(max_length=100,blank=True)
    def is_active(self):
        from django.utils import timezone
        if self.plan=='free': return True
        return self.status=='active' and (self.expires_at is None or self.expires_at>timezone.now())

# payments/views.py
import razorpay,json
PLAN_PRICES={"starter":19900,"pro":49900,"trader":99900}  # in paise
rz=razorpay.Client(auth=(settings.RAZORPAY_KEY_ID,settings.RAZORPAY_KEY_SECRET))

@login_required
def create_order(request):
    plan=json.loads(request.body).get('plan')
    order=rz.order.create({'amount':PLAN_PRICES[plan],'currency':'INR',
        'receipt':f"mp_{request.user.id}_{plan}",'notes':{'plan':plan}})
    return JsonResponse({'order_id':order['id'],'amount':PLAN_PRICES[plan],'key_id':settings.RAZORPAY_KEY_ID
})

@login_required
def payment_success(request):
    data=json.loads(request.body)
    try:
```

```
        rz.utility.verify_payment_signature({'razorpay_order_id':data['razorpay_order_id'],
            'razorpay_payment_id':data['razorpay_payment_id'],'razorpay_signature':data['razorpay_signature'
]})
    except: return JsonResponse({'error':'Verification failed'},status=400)
    from django.utils import timezone; from datetime import timedelta
    sub,_=Subscription.objects.get_or_create(user=request.user)
    plan=data.get('plan','starter')
    sub.plan=plan; sub.status='active'; sub.expires_at=timezone.now()+timedelta(days=30); sub.save()
    return JsonResponse({'success':True,'plan':plan})
```

Month	Free	Starter ■199	Pro ■499	Trader ■999	Revenue
6	500	50	10	2	■17,928
12	2,000	200	50	10	■74,800
18	8,000	600	150	30	■1,23,600
24	25,000	2,000	500	100	■6,47,800
36	1,00,000	10,000	3,000	500	■33,94,500

15. Zerodha API — Cost, Setup & Integration

Plan	Cost	Best For
Kite Connect Developer	■2,000/month	Production apps — unlimited calls
Kite Connect Partner	Revenue sharing	High volume (1000+ users)
Free Trial	■0 (30 days)	Testing only

Note: ■2,000/month fixed. Pehle 50+ paid users aane do, tab Zerodha API lo. Tab ROI positive hoga.

15.1 Zerodha OAuth Flow

1	**User clicks 'Connect Zerodha'**	Teri app mein ek button — 'Link Zerodha Account'
2	**Redirect to Zerodha login**	User Zerodha pe jaata hai — apna ID/password daalta hai
3	**User grants permission**	Zerodha: 'MarketPulse ko read access dena hai?' — User Yes
4	**Zerodha sends request_token**	Redirect back with ?request_token=xxx in URL
5	**Exchange for access_token**	request_token + API secret → access_token
6	**Fetch portfolio**	holdings(), positions(), margins() — sab data
7	**Save to DB**	UserWatchlist mein sync karo
8	**Done!**	Portfolio teri app mein live! Auto-sync daily.

15.2 Complete Zerodha Integration Code (portfolio/zerodha.py)

```
from kiteconnect import KiteConnect
from django.conf import settings
from django.core.cache import cache

def get_login_url(user_id:int)->str:
    kite=KiteConnect(api_key=settings.ZERODHA_API_KEY)
    cache.set(f"zerodha_auth_{user_id}",user_id,600); return kite.login_url()

def exchange_token(request_token:str,user)->str:
    kite=KiteConnect(api_key=settings.ZERODHA_API_KEY)
    data=kite.generate_session(request_token,api_secret=settings.ZERODHA_API_SECRET)
    access_token=data["access_token"]
    from users.models import BrokerConnection
    BrokerConnection.objects.update_or_create(user=user,broker='zerodha',
        defaults={'access_token':access_token,'is_active':True})
    return access_token

def fetch_portfolio(user)->dict:
    from users.models import BrokerConnection
    conn=BrokerConnection.objects.get(user=user,broker='zerodha',is_active=True)
```

```
    kite=KiteConnect(api_key=settings.ZERODHA_API_KEY)
    kite.set_access_token(conn.access_token)
    holdings=kite.holdings(); total_pnl=0; result=[]
    for h in holdings:
        if h['quantity']>0:
            pnl_pct=((h['last_price']-h['average_price'])/h['average_price']*100) if h['average_price'] else
 0
            result.append({'symbol':h['tradingsymbol'],'exchange':h['exchange'],
                'quantity':h['quantity'],'avg_price':round(h['average_price'],2),
                'current':round(h['last_price'],2),'pnl':round(h['pnl'],2),'pnl_pct':round(pnl_pct,2)})
            total_pnl+=h['pnl']
    return {'holdings':result,'total_pnl':round(total_pnl,2),'cash':kite.margins().get('equity',{}).get('ava
ilable',{}).get('cash',0)}

def sync_to_watchlist(user,portfolio:dict):
    from watchlist.models import Stock,UserWatchlist
    for h in portfolio['holdings']:
        stock,_=Stock.objects.get_or_create(symbol=h['symbol'],defaults={'name':h['symbol'],'exchange':h['ex
change']})
        UserWatchlist.objects.update_or_create(user=user,stock=stock,
            defaults={'quantity':h['quantity'],'avg_price':h['avg_price'],'source':'zerodha'})

# portfolio/views.py
@login_required
def zerodha_connect(request): return redirect(get_login_url(request.user.id))

@login_required
def zerodha_callback(request):
    request_token=request.GET.get('request_token')
    if not request_token or request.GET.get('status')!='success':
        return redirect('/dashboard/?zerodha=failed')
    try:
        token=exchange_token(request_token,request.user)
        portfolio=fetch_portfolio(request.user)
        sync_to_watchlist(request.user,portfolio)
        return redirect('/dashboard/?zerodha=success')
    except: return redirect('/dashboard/?zerodha=error')
```

16. PAN Card + OTP → Auto Portfolio Import

Important: PAN se directly portfolio milna legally restricted hai 2026 mein bhi. Yeh section: (1) kya legal hai, (2) best working approaches, (3) complete code.

Approach	Legal?	Works?	Cost	Recommended?
Broker OAuth (Zerodha etc)	Yes	Yes — Best	■2,000/mo	Phase 3 ✓
DigiLocker CAS Statement	Yes	Yes — All brokers	Free	Phase 3 ✓
Account Aggregator (Setu)	Yes	Yes — Future ready	Per-user	Phase 4 ✓
Manual CSV Upload	Yes	Yes — Works now	Free	Phase 1 ✓
Direct CDSL/NSDL API	Yes	Very Limited	■10L+ setup	No ✗
Screen scraping	No	Illegal	N/A	Never ✗

16.1 PAN Verify + OTP + Broker OAuth — Complete Flow

1	**User enters PAN**	Form mein PAN number — format validate karo (AAAAA1234A)
2	**PAN verify via Surepass**	■2-3 per verify — name match confirm karo
3	**Send OTP**	Fast2SMS ya Twilio se 6-digit OTP to registered phone
4	**OTP verify**	Redis mein 5-min expiry ke saath OTP store/check
5	**PAN encrypt + save**	Fernet encryption se encrypted PAN DB mein save karo
6	**Broker select**	Zerodha / CSV upload option dikhao
7	**Portfolio fetch**	Broker OAuth ya CSV parse → DB sync
8	**Done!**	Full portfolio across all brokers imported!

16.2 Complete PAN + OTP Views (portfolio/views.py)

```
import json,re,random
from django.http import JsonResponse
from django.contrib.auth.decorators import login_required
from django.core.cache import cache
import requests
from django.conf import settings

def verify_pan_format(pan:str)->bool:
    return bool(re.match(r'^[A-Z]{5}[0-9]{4}[A-Z]{1}$',pan.upper().strip()))

def verify_pan_api(pan:str)->dict:
    """Surepass API — Rs 2-3 per verify"""
    resp=requests.post("https://kyc-api.surepass.io/api/v1/pan/pan",
        headers={"Authorization":f"Bearer {settings.SUREPASS_TOKEN}","Content-Type":"application/json"},
        json={"id_number":pan.upper()})
```

```
    data=resp.json()
    if data.get('success'):
        return {'valid':True,'name':data['data'].get('full_name','')}
    return {'valid':False,'error':data.get('message','Failed')}

def encrypt_pan(pan:str)->str:
    from cryptography.fernet import Fernet
    f=Fernet(settings.PAN_ENCRYPTION_KEY.encode())
    return f.encrypt(pan.encode()).decode()

@login_required
def step1_pan_verify(request):
    data=json.loads(request.body)
    pan=data.get('pan','').upper().strip()
    if not verify_pan_format(pan):
        return JsonResponse({'success':False,'error':'Invalid PAN format (e.g. ABCDE1234F)'})
    result=verify_pan_api(pan)
    if not result['valid']:
        return JsonResponse({'success':False,'error':result['error']})
    request.session['pending_pan']=pan
    return JsonResponse({'success':True,'name':result['name'],'next':'send_otp'})

@login_required
def step2_send_otp(request):
    phone=request.user.profile.phone
    if not phone: return JsonResponse({'error':'Phone not set in profile'},status=400)
    otp=str(random.randint(100000,999999))
    cache.set(f"pan_otp_{request.user.id}",otp,300)  # 5 min
    # Fast2SMS — free 50/day
    requests.get("https://www.fast2sms.com/dev/bulkV2",
        params={'authorization':settings.FAST2SMS_API_KEY,'variables_values':otp,'route':'otp','numbers':pho
ne})
    return JsonResponse({'success':True,'message':f'OTP sent to {phone[:4]}XXXXXX'})

@login_required
def step3_verify_otp(request):
    data=json.loads(request.body)
    otp_input=data.get('otp','')
    stored=cache.get(f"pan_otp_{request.user.id}")
    if not stored or otp_input!=stored:
        return JsonResponse({'success':False,'error':'Wrong OTP. Try again.'})
    pan=request.session.get('pending_pan')
    if pan:
        profile=request.user.profile
        profile.pan_number=encrypt_pan(pan); profile.pan_verified=True
        from django.utils import timezone; profile.pan_verified_at=timezone.now()
        profile.save()
        del request.session['pending_pan']
        cache.delete(f"pan_otp_{request.user.id}")
    return JsonResponse({'success':True,'pan_saved':True,'next':'connect_broker'})

@login_required
def step4_csv_upload(request):
    """Fallback: User uploads CSV exported from their broker"""
    if request.method!='POST': return JsonResponse({'error':'POST only'})
    csv_file=request.FILES.get('portfolio_csv')
    broker=request.POST.get('broker','generic')
    if not csv_file: return JsonResponse({'error':'No file'})
    import csv,io
    from watchlist.models import Stock,UserWatchlist
    cols={'zerodha':{'s':'Instrument','q':'Qty.','p':'Avg. cost'},
          'groww':{'s':'Symbol','q':'Quantity','p':'Average Price'},
          'generic':{'s':'Symbol','q':'Quantity','p':'Average Price'}}
    c=cols.get(broker,cols['generic']); imported=0
    for row in csv.DictReader(io.StringIO(csv_file.read().decode('utf-8'))):
        try:
```

```
            sym=row.get(c['s'],'').strip(); qty=int(float(row.get(c['q'],0))); price=float(row.get(c['p'],0)
)
            if not sym or qty<=0: continue
            stock,_=Stock.objects.get_or_create(symbol=sym,defaults={'name':sym})
            UserWatchlist.objects.update_or_create(user=request.user,stock=stock,
                defaults={'quantity':qty,'avg_price':price,'source':broker})
            imported+=1
        except: continue
    return JsonResponse({'success':True,'imported':imported,'message':f'{imported} stocks imported!'})
```

PART F: Operations & Growth

17. Database Schema Reference

Model	App	Key Fields	Relations
NewsSource	news	name, url, category, credibility_score	-
NewsArticle	news	title, url, summary, published, is_analyzed	FK NewsSource
SentimentResult	sentiment	sentiment, confidence, importance, stocks[], sectors[]	1-1 NewsArticle
Stock	watchlist	symbol, name, exchange, sector	-
UserWatchlist	watchlist	quantity, avg_price, alert_settings, source	FK User, FK Stock
TradeDecision	portfolio	action, price, reason, emotion, ai_review	FK User
UserProfile	users	telegram_id, phone, pan_number(enc), risk_appetite	1-1 User
Subscription	users	plan, status, expires_at, razorpay_sub_id	1-1 User
BrokerConnection	users	broker, access_token(enc), is_active	FK User
PaymentHistory	payments	amount, plan, razorpay_order, status	FK User
AlertLog	alerts	channel, status, sent_at	FK User, FK SentimentResult

```
python manage.py makemigrations news sentiment watchlist alerts portfolio users payments
python manage.py migrate
python manage.py createsuperuser
```

18. Celery Configuration

```
# marketpulse/celery.py
import os
from celery import Celery
os.environ.setdefault('DJANGO_SETTINGS_MODULE','marketpulse.settings')
app=Celery('marketpulse')
app.config_from_object('django.conf:settings',namespace='CELERY')
app.autodiscover_tasks()

# marketpulse/__init__.py
from .celery import app as celery_app
__all__=('celery_app',)

# Local dev — 4 terminals:
```

```
# 1: redis-server
# 2: celery -A marketpulse worker --loglevel=info --concurrency=4
# 3: celery -A marketpulse beat --loglevel=info
# 4: python manage.py runserver
# Monitor: celery -A marketpulse flower   (localhost:5555)
```

19. Frontend & Dashboard

```
# dashboard/views.py
from django.views.generic import TemplateView
from django.contrib.auth.mixins import LoginRequiredMixin
class DashboardView(LoginRequiredMixin,TemplateView):
    template_name='dashboard/index.html'
    def get_context_data(self,**kwargs):
        ctx=super().get_context_data(**kwargs)
        from watchlist.models import UserWatchlist
        from sentiment.models import SentimentResult
        watchlist=list(UserWatchlist.objects.filter(user=self.request.user).values_list('stock__symbol',flat
=True))
        all_art=SentimentResult.objects.filter(importance__gte=5).select_related('article','article__source'
).order_by('-analyzed_at')[:100]
        ctx.update({'relevant_news':[a for a in all_art if any(s in a.stocks for s in watchlist)][:15],
            'general_news':[a for a in all_art][:20],'watchlist':watchlist,
            'bullish_today':SentimentResult.objects.filter(sentiment='bullish',importance__gte=7).count(),
            'bearish_today':SentimentResult.objects.filter(sentiment='bearish',importance__gte=7).count()})
        return ctx
# marketpulse/urls.py
urlpatterns=[path('admin/',admin.site.urls),path('',DashboardView.as_view(),name='dashboard'),
    path('accounts/',include('allauth.urls')),path('api/chat/',ai_chat,name='ai_chat'),
    path('api/earnings/',earnings_analyzer),path('portfolio/',include('portfolio.urls')),
    path('watchlist/',include('watchlist.urls')),path('payments/',include('payments.urls')),
    path('zerodha/connect/',zerodha_connect),path('zerodha/callback/',zerodha_callback)]
```

20. Deployment on Railway.app

```
# Procfile
web: gunicorn marketpulse.wsgi --workers 2 --bind 0.0.0.0:$PORT --timeout 120
worker: celery -A marketpulse worker --loglevel=info --concurrency=2
beat: celery -A marketpulse beat --loglevel=info
release: python manage.py migrate --no-input

# .env — NEVER commit to GitHub!
SECRET_KEY=your-50-char-secret-key
DEBUG=False
ALLOWED_HOSTS=your-app.railway.app,marketpulse.ai
DATABASE_URL=postgresql://user:pass@host:5432/marketpulse
REDIS_URL=redis://default:pass@redis.railway.internal:6379
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxx
NEWS_API_KEY=your-newsapi-key
TELEGRAM_BOT_TOKEN=1234567890:ABCxxxxx
EMAIL_USER=alerts@gmail.com; EMAIL_PASS=gmail-app-password
ZERODHA_API_KEY=your-key; ZERODHA_API_SECRET=your-secret
RAZORPAY_KEY_ID=rzp_live_xxx; RAZORPAY_KEY_SECRET=your-secret
SUREPASS_TOKEN=your-token; FAST2SMS_API_KEY=your-key
PAN_ENCRYPTION_KEY=your-32-byte-fernet-key
```

Checklist Item	Priority	Status
DEBUG=False	Critical	Set in .env
HTTPS/SSL	Auto	Railway handles it
Static files whitenoise	Required	In settings.py
PostgreSQL connected	Required	Railway DB plugin
Redis connected	Required	Railway Redis plugin
Celery worker running	Required	In Procfile
Sentry error tracking	Recommended	sentry.io free tier
Privacy policy page	Legal req.	Add before launch

21. Monetization & Revenue

Month	Free	Starter ■199	Pro ■499	Trader ■999	Monthly Revenue	API Cost
6	500	50	10	2	■17,928	■1,100
12	2,000	200	50	10	■74,800	■3,000
18	8,000	600	150	30	■1,23,600	■6,000
24	25,000	2,000	500	100	■6,47,800	■15,000
36	1,00,000	10,000	3,000	500	■33,94,500	■50,000

Tip: Month 36 pe: Revenue ■33.9L/month, API cost ■50k/month. Net margin ~98%. Pure SaaS magic!

22. 30-Week Roadmap

Weeks	Goal	Key Deliverables	Success Metric
1-2	Project Setup	Django, apps, PostgreSQL, Redis	App runs locally
3-4	News Pipeline	5+ RSS feeds, articles in DB	100 articles/day
5-6	AI Sentiment	Claude batch analysis working	All articles tagged
7-8	Basic Dashboard	News feed UI, sentiment badges	Usable dashboard
9-10	User Auth	Login, signup, profile	User can login
11-12	Personalization	News filtered by watchlist	Personal feed works
13-14	Alerts V1	Telegram bot + email	Alerts delivered
15-16	Morning Digest	7am automated digest	Daily digest sent
17-18	AI Chatbot	Hinglish chatbot live	Chat working
19-20	FOMO Engine	3%+ move alerts with AI	FOMO alerts sent
21-22	Rate Limiting	Redis limits per plan	Limits enforced
23-24	Razorpay	Payment + subscription	Payment working
25-26	Zerodha OAuth	Portfolio import live	Portfolio synced
27-28	PAN + OTP Flow	Pan verify + CSV upload	PAN flow working
29-30	Deploy + Launch	Railway, domain, beta users	50 beta users!

Note: Ek week, ek cheez. Perfect execution > rushing everything. Ship karo, seekhte jao.

23. Legal & Compliance

Important: Ye sab non-negotiable hain. SEBI aur RBI fintech pe strict hain.

What You Offer	SEBI Status	Action Required
News + AI sentiment (general info)	Not regulated	Add standard disclaimer
Portfolio tracking	Not regulated	Safe — just viewing
Tax calculator	Not regulated	Safe — just computation
AI earnings analysis (general)	Not regulated (info only)	Add 'not advice' disclaimer
Paid stock recommendations	Research Analyst (RA) license	Register with SEBI if doing this
Buy/Sell recommendations	Investment Advisor (IA)	Mandatory SEBI registration
Algo trading signals	Complex licensing	Avoid in MVP completely

- **PAN encryption:** Fernet AES-256 — plaintext kabhi nahi
- **Access tokens:** Zerodha/broker tokens bhi encrypt karo
- **DPDP Act 2023:** India ka new privacy law — follow karo
- **Disclaimer:** Har page pe 'Yeh investment advice nahi hai' lagao
- **SEBI disclaimer:** Har AI response ke end mein add karo
- **Privacy policy:** Launch se pehle zaroor banao

```
# templates/base.html — Footer mein ZAROOR:
DISCLAIMER = "MarketPulse AI ek information platform hai.
Iske content ko investment advice ya stock recommendation ke roop mein
interpret mat karein. Koi bhi investment decision lene se pehle apne
SEBI-registered financial advisor se salah lein.
Stock market mein investments market risks ke adheen hain."

# Har AI response ke end mein:
AI_DISCLAIMER = "Yeh sirf information hai, investment advice nahi. Market risk hota hai."
```

Tera Complete Blueprint Ready Hai!

Ek PDF. Sab kuch. Ab sirf build karna baaki hai.

Week 1-2 se shuru karo. Jab bhi atak jao — main hoon yahan!

MarketPulse AI | Built with Claude | anthropic.com